

**videncia de producto: GA8-220501096-AA2-EV02 - APK (desarrollar módulos móvil
según requerimientos del proyecto)**

ANÁLISIS Y DESARROLLO DE SOFTWARE

Ficha (2977481)

Autor(es): Hader Borré

Dirigido a:

Milton Barbosa

29/09/2025

Introducción

El avance de la tecnología y la creciente necesidad de soluciones remotas han impulsado el desarrollo de sistemas que optimicen la atención al usuario en el ámbito del soporte técnico. En este contexto, el proyecto Nexa Support surge como una alternativa innovadora que combina la gestión de usuarios, la administración de servicios, la asignación de técnicos especializados y la generación de notificaciones, todo ello soportado en una arquitectura cliente-servidor desarrollada en Apache NetBeans e integrada con una base de datos MySQL.

La finalidad principal de este proyecto es brindar un sistema robusto que permita a los clientes registrar solicitudes de soporte de manera sencilla y eficiente, al mismo tiempo que los técnicos disponen de herramientas para gestionar sus asignaciones, disponibilidad y especialidad. La inclusión de tablas como usuarios, tecnicos, servicios, tickets y notificaciones asegura una correcta organización de la información, favoreciendo la trazabilidad y transparencia en los procesos de soporte.

El uso de Apache NetBeans como entorno de desarrollo facilita la implementación de funcionalidades avanzadas mediante Java, JSP y Servlets, mientras que la conexión con MySQL garantiza persistencia de datos confiable. Asimismo, el diseño del sistema se fundamenta en principios de modularidad, escalabilidad y seguridad, lo que asegura su adaptabilidad a diferentes contextos empresariales.

De esta manera, el proyecto no solo responde a una necesidad académica de integración de herramientas, sino que además plantea un modelo real de cómo optimizar procesos de atención al cliente en el mundo actual, donde la inmediatez y la eficiencia son factores clave.

Objetivo General

Desarrollar un sistema de soporte remoto basado en Apache NetBeans y MySQL que permita la gestión integral de usuarios, técnicos, servicios, tickets y notificaciones, con el fin de mejorar la eficiencia en la atención de incidencias técnicas y fortalecer la comunicación entre cliente y proveedor de soporte.

Objetivos Específicos

1. Diseñar y normalizar la base de datos en MySQL, estructurando tablas que representen usuarios, técnicos, servicios, tickets y notificaciones con sus respectivas relaciones.
2. Implementar en NetBeans un entorno de conexión entre la aplicación y la base de datos utilizando el conector JDBC, garantizando seguridad y eficiencia en el acceso a los datos.
3. Desarrollar interfaces gráficas y funcionalidades que permitan el registro, consulta, actualización y eliminación de información relacionada con usuarios, servicios y tickets.
4. Integrar un módulo de notificaciones que facilite la comunicación al cliente sobre el estado de sus solicitudes y la asignación de técnicos.
5. Aplicar buenas prácticas de seguridad, como el uso de contraseñas cifradas y validaciones, para garantizar la integridad de los datos y la protección de los usuarios.
6. Evaluar el desempeño del sistema mediante pruebas funcionales y de conectividad, validando la correcta interacción entre los módulos implementados.

Componentes esenciales de una aplicación Android

Una aplicación Android se compone principalmente de los siguientes elementos:

1. **Activities** → pantallas de la aplicación (interfaces gráficas).
2. **Services** → procesos que se ejecutan en segundo plano (ejemplo: música, descargas).
3. **Broadcast Receivers** → permiten que la app responda a mensajes o eventos del sistema (ejemplo: batería baja, llegada de un SMS).
4. **Content Providers** → manejan y comparten datos entre aplicaciones (ejemplo: lista de contactos).
5. **Manifest (AndroidManifest.xml)** → archivo central donde se definen permisos, actividades principales y configuraciones de la app.

Sección 2 Taller

Crear la primera aplicación Android.

Paso a Paso – Creación de la aplicación *Nexa Support* en MIT App Inventor

Paso 1. Acceso y creación del proyecto

1. Ingresé a MIT App Inventor.
2. Inicié sesión con mi cuenta de Google.
3. Fui al menú **Proyectos > Iniciar un nuevo proyecto**.
4. Asigné el nombre: **Nexa_Support**.

Este nombre identifica el proyecto dentro de App Inventor.

Paso 2. Configuración de la pantalla principal (*Screen1*)

1. MIT App Inventor crea por defecto una pantalla llamada **Screen1**.
2. En el panel derecho configuré:
 - **Título:** “Nexa support”.
 - **Color de la barra superior:** azul corporativo.
 - **Orientación:** vertical (para móviles Android).
 - **Visible:** verdadero.

Esta pantalla será la **pantalla de inicio de sesión** de la aplicación.

Paso 3: Creación de la interfaz gráfica

Desde la sección **Interfaz de usuario** arrastré y solté componentes dentro del simulador de teléfono:

1. Etiqueta de título

- Texto: **“Nexa support”**.
- Fuente: en negrita y centrada.

2. Etiqueta secundaria

- Texto: **“Soporte remoto eficiente y confiable”**.
- Función: servir como slogan.

3. Imagen

- Subí un recurso gráfico (fondo_web.png) con el robot y diseño tecnológico.
- Inserté el componente **Imagen** y lo vinculé al archivo.

4. Campos de texto

- Arrastré dos **CamposDeContraseña**.
- El primero para el **usuario/correo**.
- El segundo para la **contraseña**.
- Configuré Hint (texto de sugerencia): “Usuario” y “Contraseña”.

5. Interruptor

- Inserté un interruptor llamado **Mostrar_contraseña**.
- Permite alternar la visibilidad de la contraseña.

6. Botón de acceso

- Texto: **“Iniciar sesión”**.
- Color: azul con letras blancas.

7. Enlaces secundarios

- Texto 1: **“¿No eres cliente aún? Regístrate”**.
- Texto 2: **“¿Olvidaste tu contraseña?”**.
- Cada uno se configuró como **Etiquetas** con formato subrayado y color azul, simulando enlaces.

8. Pie de página

- Texto: “© 2025 Nexa Support. Todos los derechos reservados.”
- Ubicación: parte inferior de la pantalla.

Paso 4: Organización de los componentes

Para alinear bien los elementos utilicé **disposiciones horizontales y verticales** (Layout):

- Un **VerticalArrangement** principal, donde coloqué todos los elementos de arriba a abajo.
- Dentro de él, **HorizontalArrangement** para alinear los enlaces en la parte inferior.

Paso 5: Configuración de propiedades

En el panel derecho personalicé propiedades de cada componente:

- **Color de texto y de fondo.**
- **Tamaño de letra.**
- **Visibilidad inicial.**
- Para el campo de contraseña, marqué la casilla de “**Solo números**” como **falso**, permitiendo cualquier carácter.

Paso 6: Programación con bloques

1. Fui a la vista **Bloques**.
2. Creé la lógica básica del login:
 - Si el usuario y la contraseña coinciden con valores definidos (ejemplo: usuario: admin, contraseña: 12345), entonces mostrar un **Notificador** con el mensaje “**Bienvenido a Nexa Support**”.
 - En caso contrario, mostrar mensaje “**Credenciales incorrectas**”.
3. Configuré el bloque del interruptor **Mostrar_contraseña**:
 - Si está activado, el campo de contraseña se muestra en texto normal.
 - Si está desactivado, vuelve al formato oculto (asteriscos).

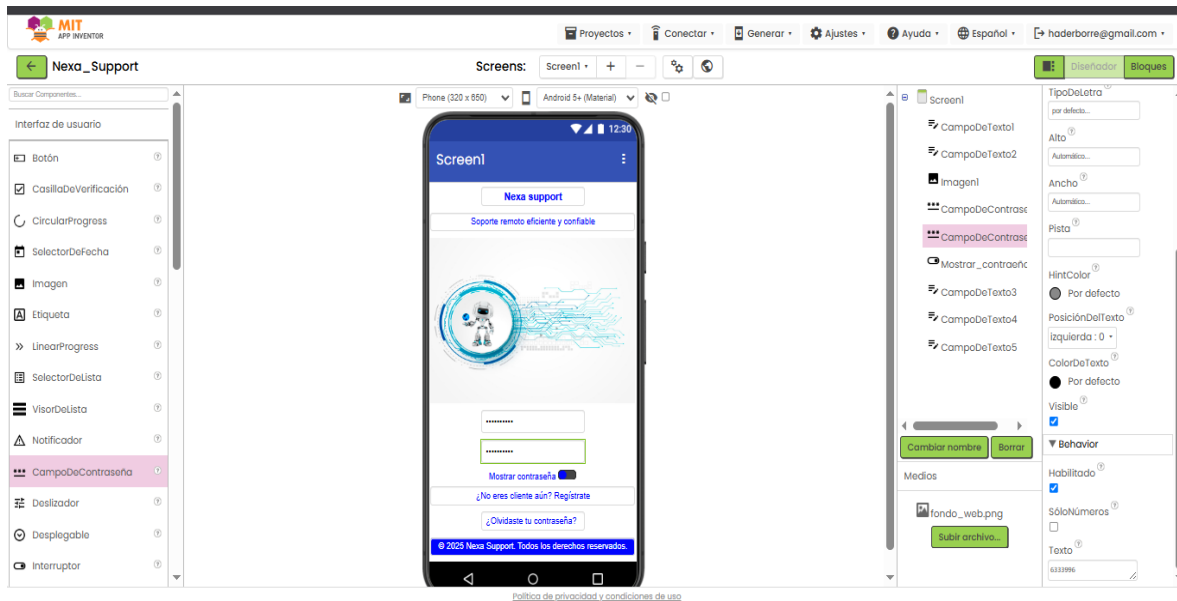
4. Vinculé los botones/enlaces secundarios a mensajes de alerta simulados (por ejemplo: “Función de registro en construcción”).

7. Pruebas en dispositivo/emulador

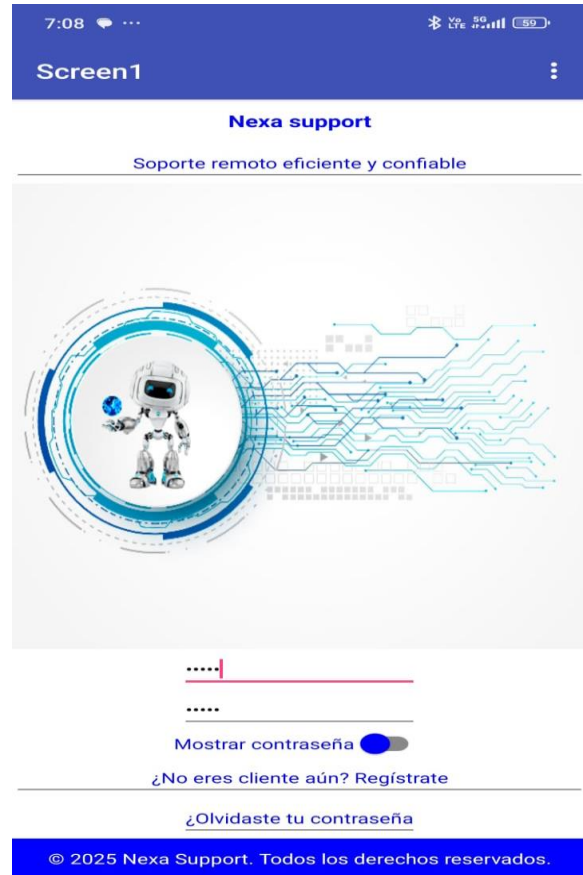
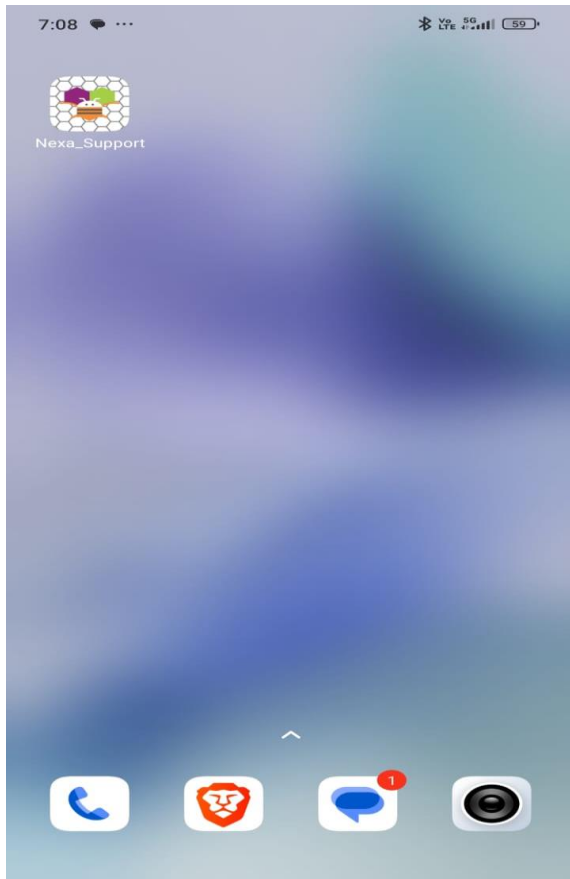
1. Desde el menú **Conectar > AI Companion**, escaneé el código QR con mi celular Android.
2. Probé en tiempo real el login, la visibilidad de contraseña y los botones de registro/recuperar clave.
3. Ajusté tamaños de texto y colores hasta que la interfaz quedó clara y legible.

8. Generación del APK

1. Una vez lista la aplicación, fui al menú **Generar > App (APK)**.
2. Guardé el archivo APK en mi computadora.
3. Lo instalé en mi dispositivo Android para comprobar el funcionamiento final.



Instalación en mi dispositivo nexa support



Requisitos funcionales y no funcionales del aplicativo

Requisitos Funcionales

Gestión de Usuarios

Registro y Autenticación

- **RF-01:** El sistema debe permitir a los técnicos registrarse con un nombre de usuario y una contraseña.
- **RF-02:** El sistema debe permitir a los técnicos autenticarse usando su nombre de usuario y contraseña.

Gestión de Roles

- **RF-03:** El sistema debe permitir asignar roles a los usuarios (e.g., administrador, técnico).
-

Conexión Remota

Establecimiento de Conexión

- **RF-04:** El sistema debe permitir a los técnicos solicitar conexión remota a la computadora del cliente.
- **RF-05:** El sistema debe permitir a los clientes aceptar o rechazar solicitudes de conexión remota.

acceso Remoto

- **RF-06:** El sistema debe permitir a los técnicos controlar la computadora de la cliente una vez establecida la conexión.
- **RF-07:** El sistema debe permitir a los clientes interrumpir la conexión remota en cualquier momento.

Diagnóstico y Solución de Problemas

- **RF-08:** El sistema debe permitir a los técnicos ejecutar diagnósticos en la computadora del cliente.
- **RF-09:** El sistema debe permitir a los técnicos realizar tareas de mantenimiento y solucionar problemas en la computadora del cliente.

Transferencia de Archivos

- **RF-10:** El sistema debe permitir la transferencia segura de archivos entre el técnico y el cliente.

Comunicación

- **RF-11:** El sistema debe incluir un chat en tiempo real para la comunicación entre técnicos y clientes durante la sesión remota.
- **RF-12:** El sistema debe permitir videollamadas entre técnicos y clientes.

Requisitos No Funcionales

Rendimiento

- **RNF-01:** El sistema debe ser capaz de manejar hasta 100 conexiones remotas simultáneas sin degradación significativa del rendimiento.
- **RNF-02:** El tiempo de respuesta para establecer una conexión remota no debe exceder los 10 segundos.

Seguridad

- **RNF-03:** Todas las comunicaciones y transferencias de archivos deben estar encriptadas utilizando TLS.
- **RNF-04:** El sistema debe cumplir con las normativas de protección de datos aplicables (GDPR).

Usabilidad

- **RNF-05:** La interfaz de usuario debe ser intuitiva y fácil de usar tanto para técnicos como para clientes.
- **RNF-06:** El sistema debe proporcionar documentación y tutoriales para los usuarios.

Disponibilidad

- **RNF-07:** El sistema debe estar disponible al menos el 99.9% del tiempo, excluyendo períodos de mantenimiento programado.

Escalabilidad

- **RNF-08:** El sistema debe ser escalable para soportar un incremento en el número de usuarios y conexiones remotas.

Mantenibilidad

- **RNF-09:** El sistema debe estar diseñado de manera modular para facilitar el mantenimiento y la actualización.

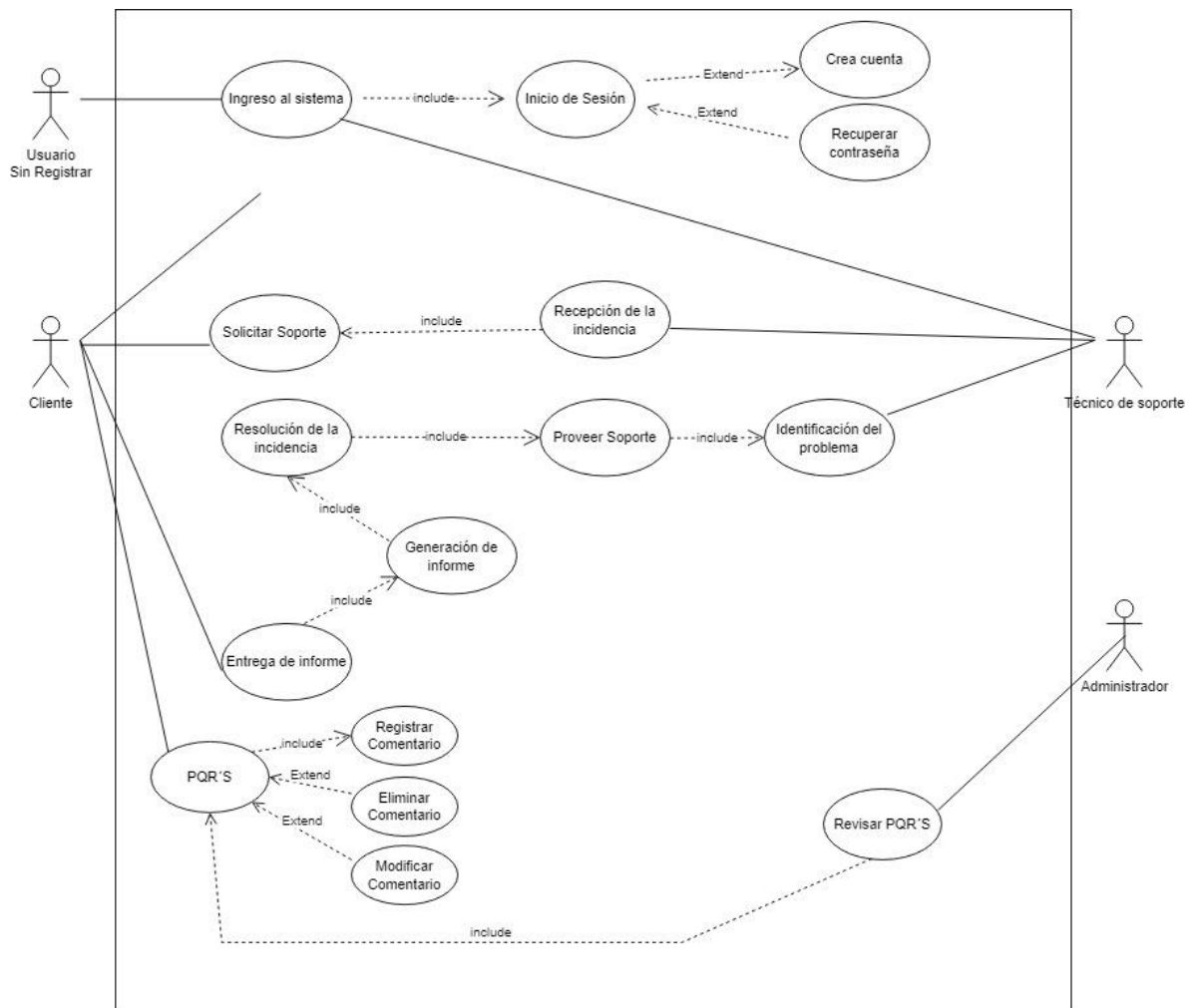
Compatibilidad

- **RNF-10:** El sistema debe ser compatible con los principales sistemas operativos (e.g., Windows, macOS, Linux)

Historia de usuarios

ID de la historia	Rol	Funcionalidad	Razón/Resultado	Criterio de aceptación
Id.01	Como técnico	Quiero poder registrarme en el sistema.	Para poder acceder a la plataforma y brindar soporte técnico.	El sistema permite el registro de técnicos con la información requerida y la creación de una cuenta personal.
Id.02	Como técnico	Quiero poder iniciar sesión en el sistema.	Para acceder a las funcionalidades de soporte técnico.	El sistema permite la autenticación con credenciales válidas y redirige a la interfaz principal de soporte.
Id.03	Como técnico	Quiero poder solicitar la conexión remota.	Para diagnosticar y solucionar problemas sin estar físicamente presente.	El sistema permite enviar solicitudes de conexión remota con notificaciones al cliente.
Id.04	Como cliente	Quiero aceptar o rechazar la solicitud de conexión remota.	Para garantizar que el acceso a mi equipo sea con mi consentimiento.	El sistema permite aceptar o rechazar solicitudes de conexión con confirmación visual del estado.
Id.05	Como técnico	Quiero poder controlar remotamente el equipo del cliente.	Para realizar configuraciones y reparaciones en el sistema.	El sistema permite el control remoto completo del equipo del cliente con acceso a herramientas de soporte.
Id.06	Como técnico o cliente	Quiero poder interrumpir la conexión remota.	Para finalizar o detener la sesión de manera controlada.	El sistema permite interrumpir la conexión con confirmación visual del cierre seguro de la sesión.
Id.07	Como técnico	Quiero poder ejecutar herramientas de diagnóstico durante la conexión.	Para identificar y comprender los problemas técnicos.	El sistema permite la ejecución de diagnósticos y muestra los resultados de manera clara para su análisis.
Id.08	Como técnico	Quiero poder implementar soluciones a los problemas detectados.	Para resolver los incidentes reportados y asegurar el correcto funcionamiento.	El sistema permite la aplicación de soluciones y actualiza el estado de soporte como resuelto.
Id.09	Como técnico o cliente	Quiero poder transferir archivos de manera segura durante la sesión.	Para compartir información o herramientas sin comprometer la seguridad.	El sistema permite la transferencia de archivos cifrados y confirma la recepción de los mismos por el destinatario.

Diagrama Caso de uso



Caso de uso

El diagrama de casos de uso presentado representa las principales interacciones que se desarrollan entre los **usuarios** y el **sistema de soporte técnico remoto**. Este modelo permite visualizar qué funcionalidades ofrece la aplicación y qué actores están involucrados en cada proceso.

Actores principales

1. Usuario sin registrar

- Puede acceder al sistema únicamente para crear una cuenta o recuperar su contraseña.
- Sus acciones iniciales están ligadas al **inicio de sesión** con los casos extendidos de “**Crear cuenta**” y “**Recuperar contraseña**”.

2. Cliente

- Es el actor central que utiliza la plataforma para **solicitar soporte**.
- Una vez autenticado, puede reportar incidencias, consultar soluciones, recibir informes y gestionar el módulo de **PQR's (Petición, Quejas y Reclamos)**.
- Sus casos de uso principales incluyen:
 - **Solicitar soporte** (incluye la **recepción de la incidencia**).
 - **Revisar resolución de incidencias**, lo cual está asociado al soporte provisto por el técnico.
 - **Entrega de informe**, que documenta la solución al problema.
 - **PQR's**, donde el cliente puede registrar, modificar o eliminar comentarios sobre el servicio recibido.

3. Técnico de soporte

- Encargado de recibir los tickets de incidencias reportados por los clientes.
- Sus funciones incluyen:
 - **Recepción de la incidencia**.
 - **Identificación del problema**.
 - **Proveer soporte** al cliente para resolver el inconveniente.
 - Colaborar en la **resolución de la incidencia** y la **generación de informes**.

4. Administrador

- Tiene un rol más orientado a la supervisión.
- Su caso de uso principal es **revisar las PQR's**, asegurando la calidad del servicio y el seguimiento de los reclamos de los clientes.

Relación entre casos de uso

- El **inicio de sesión** es un requisito común para que tanto clientes como técnicos puedan ingresar al sistema.
- El caso **“Solicitar soporte”** incluye la acción de **“Recepción de la incidencia”**, ya que es indispensable registrar el problema en el sistema.
- La **resolución de incidencias** está vinculada al soporte dado por el técnico y genera un **informe** como evidencia del proceso.
- El módulo de **PQR's** tiene relaciones de extensión porque puede incluir la opción de **registrar, eliminar o modificar comentarios** dependiendo de la acción del cliente.

Importancia del modelo

Este diagrama de casos de uso:

- Ayuda a **identificar claramente las funciones clave** del sistema de soporte remoto.
- Define qué puede hacer cada actor, evitando ambigüedades en los requisitos.
- Sirve como base para el diseño técnico y funcional de la plataforma.
- Garantiza que el sistema cumpla con la **gestión de incidencias**, el **seguimiento de soporte** y la **retroalimentación de los clientes**.

Apache NetBeans

Apache NetBeans es un Entorno de Desarrollo Integrado (IDE) de código abierto que permite desarrollar aplicaciones en diferentes lenguajes de programación, principalmente Java, pero también soporta HTML, CSS, JavaScript, PHP, C/C++ y otros. Su interfaz facilita la creación de proyectos al integrar en una sola herramienta:

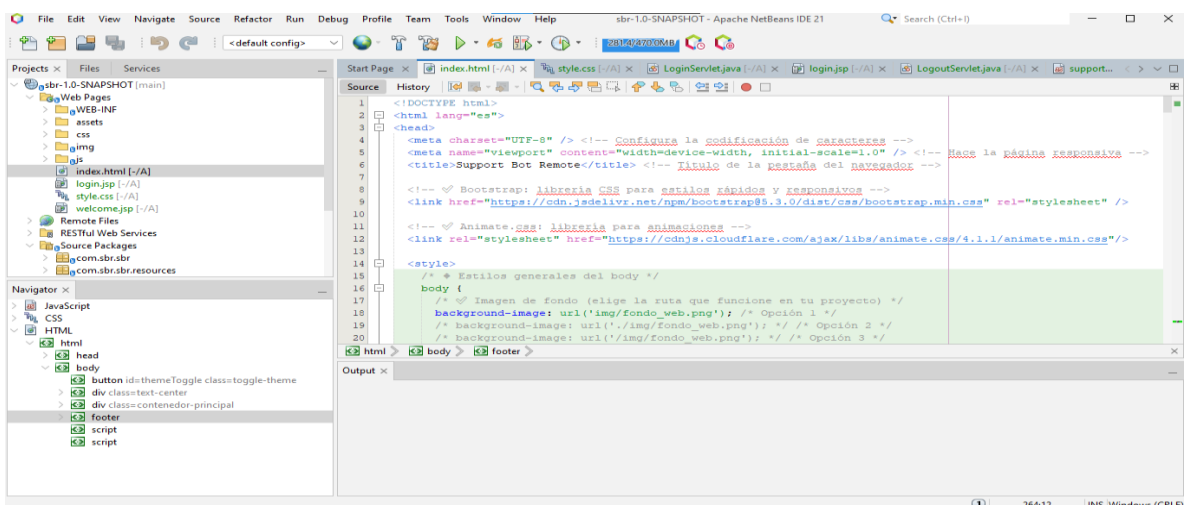
- Editor de código con autocompletado y resaltado de sintaxis.
- Depurador para encontrar y corregir errores.
- Consola de ejecución integrada.
- Administrador de proyectos para organizar archivos y carpetas.

En la imagen se observa cómo NetBeans organiza el proyecto Support Bot Remote en carpetas (Web Pages, Source Packages, etc.), permitiendo trabajar con archivos HTML, CSS, JSP y Java de forma estructurada.

Ventajas de Apache NetBeans IDE

1. Multiplataforma: funciona en Windows, Linux y macOS.
2. Soporte para múltiples lenguajes: no se limita a Java, también incluye HTML, CSS, JavaScript, PHP, entre otros.
3. Interfaz intuitiva: facilita la organización de proyectos grandes gracias a su vista de árbol.
4. Depuración eficiente: permite ejecutar programas paso a paso y detectar errores fácilmente.
5. Integración con frameworks: soporta Spring, Java EE, Hibernate, Maven, etc.
6. Compatibilidad con servidores: se conecta fácilmente con GlassFish, Apache Tomcat y Payara, entre otros.
7. Extensiones y plugins: ofrece herramientas adicionales que pueden instalarse según las necesidades del proyecto.
8. Soporte para bases de datos: permite conectarse y gestionar directamente bases de datos MySQL, PostgreSQL, Oracle, etc.
9. Comunidad activa: al ser un proyecto de código abierto, cuenta con actualizaciones frecuentes y buena documentación.

Apache NetBeans



Apache NetBeans es un Entorno de Desarrollo Integrado (IDE) de código abierto que permite desarrollar aplicaciones en diferentes lenguajes de programación, principalmente Java, pero también soporta HTML, CSS, JavaScript, PHP, C/C++ y otros. Su interfaz facilita la creación de proyectos al integrar en una sola herramienta:

- Editor de código con autocompletado y resaltado de sintaxis.
- Depurador para encontrar y corregir errores.
- Consola de ejecución integrada.
- Administrador de proyectos para organizar archivos y carpetas.

En la imagen se observa cómo NetBeans organiza el proyecto Support Bot Remote en carpetas (Web Pages, Source Packages, etc.), permitiendo trabajar con archivos HTML, CSS, JSP y Java de forma estructurada.

Ventajas de Apache NetBeans IDE

1. Multiplataforma: funciona en Windows, Linux y macOS.
2. Soporte para múltiples lenguajes: no se limita a Java, también incluye HTML, CSS, JavaScript, PHP, entre otros.
3. Interfaz intuitiva: facilita la organización de proyectos grandes gracias a su vista de árbol.
4. Depuración eficiente: permite ejecutar programas paso a paso y detectar errores fácilmente.
5. Integración con frameworks: soporta Spring, Java EE, Hibernate, Maven, etc.
6. Compatibilidad con servidores: se conecta fácilmente con GlassFish, Apache Tomcat y Payara, entre otros.
7. Extensiones y plugins: ofrece herramientas adicionales que pueden instalarse según las necesidades del proyecto.
8. Soporte para bases de datos: permite conectarse y gestionar directamente bases de datos MySQL, PostgreSQL, Oracle, etc.
9. Comunidad activa: al ser un proyecto de código abierto, cuenta con actualizaciones frecuentes y buena documentación.

Diagrama de clases

El diseño de un sistema de soporte técnico remoto requiere una estructura bien definida para gestionar eficientemente las interacciones entre clientes y técnicos. El diagrama de clases presentado proporciona una visión detallada de los principales componentes del sistema y sus relaciones, permitiendo establecer un marco claro para la implementación del software.

Este modelo organiza los elementos esenciales en torno a entidades clave como Usuario, Técnico, Cliente, Sesión Remota, Diagnóstico, Archivo y Rol. Cada una de estas clases desempeña un papel fundamental en el flujo de trabajo del soporte técnico remoto, asegurando que las solicitudes de asistencia sean gestionadas de manera estructurada y eficiente.

El sistema permite que los clientes puedan iniciar sesión y solicitar asistencia técnica, mientras que los técnicos tienen la capacidad de diagnosticar y resolver problemas de forma remota. La funcionalidad de las sesiones remotas posibilita el acceso controlado a los dispositivos de los clientes, lo que facilita la identificación y solución de fallos sin necesidad de una visita presencial. Además, se incluyen mecanismos de seguridad para la gestión de archivos y permisos, asegurando la integridad de los datos y el control sobre el acceso a la información.

Este modelo de clases proporciona una base sólida para el desarrollo del software, permitiendo escalar y mejorar el sistema con funcionalidades adicionales en el futuro.

Diagrama de clases

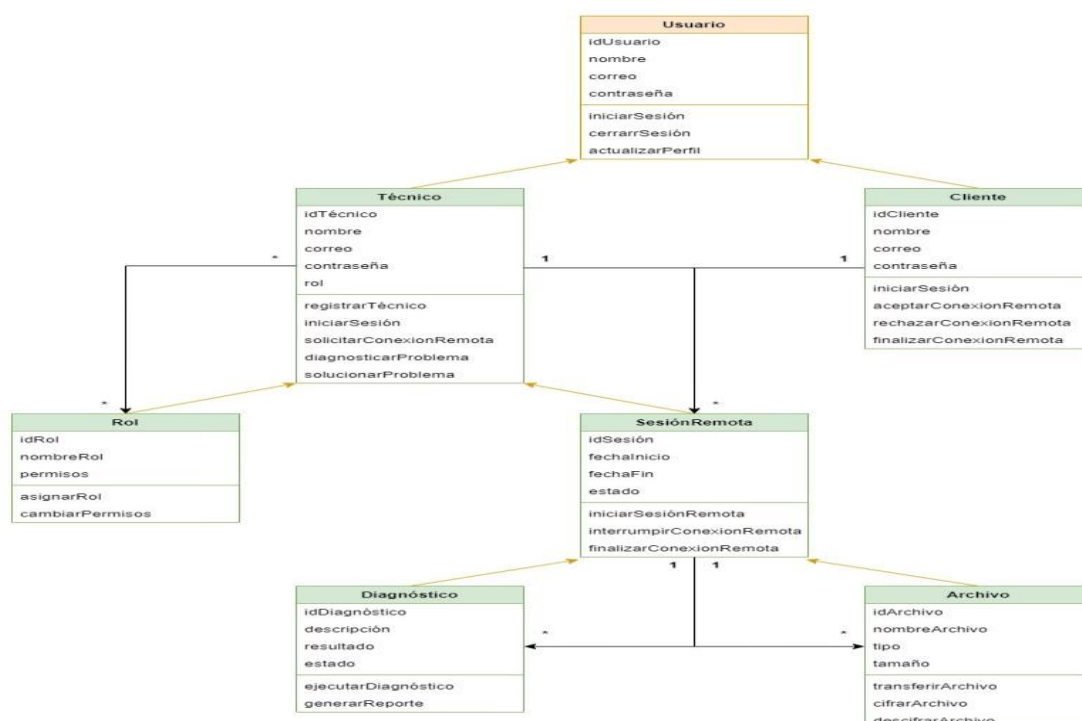
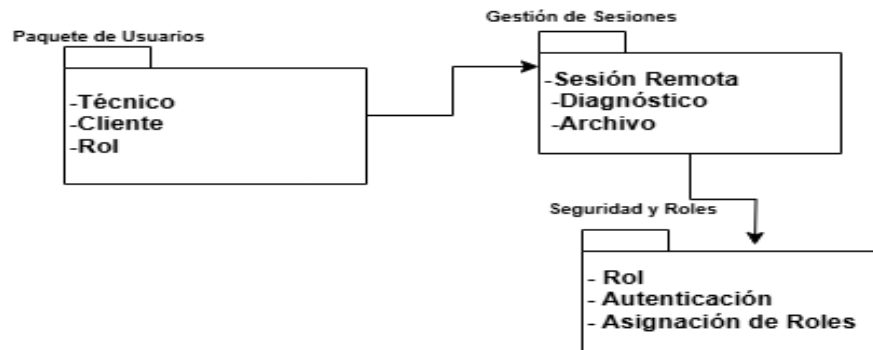


Diagrama de paquetes



El diagrama de paquetes representa la organización lógica del sistema en módulos, agrupando las clases según su funcionalidad.

1. Paquete Usuarios

- Incluye las clases Técnico, Cliente y Rol, que representan los diferentes tipos de usuarios que interactúan con el sistema.
- Se encarga de gestionar la información básica de los actores principales.

2. Paquete Gestión de Sesiones

- Contiene las clases SesiónRemota, Diagnóstico y Archivo.
- Su función es administrar las sesiones de soporte técnico, realizar diagnósticos y manejar archivos compartidos durante el soporte remoto.

3. Paquete Seguridad y Roles

- Incluye la clase Rol y define las reglas de autenticación y asignación de roles.
- Garantiza que los usuarios accedan únicamente a las funcionalidades permitidas según su perfil.

Relaciones entre los paquetes

- El paquete Usuarios depende del paquete Seguridad y Roles, ya que requiere autenticación para validar el acceso.
- El paquete Gestión de Sesiones depende tanto de Usuarios como de Seguridad, porque las sesiones son iniciadas por usuarios que deben estar previamente autenticados y con permisos asignados.

Diagrama de Componentes

El presente diagrama de componentes representa la arquitectura general de un sistema de soporte técnico remoto, detallando los módulos principales, su funcionamiento y las interacciones que se generan entre los distintos elementos. El objetivo del sistema es brindar soporte en tiempo real a los usuarios, garantizando seguridad, control de incidentes y comunicación eficiente.

Interfaz de usuario

La interfaz de usuario es el punto inicial de contacto para el cliente. A través de ella, los usuarios pueden autenticarse e iniciar sesión en el sistema. La gestión de las sesiones se realiza mediante un controlador de sesiones, lo cual permite validar y administrar la permanencia del usuario dentro de la aplicación.

Módulo de comunicación

Este componente es esencial para la conexión entre el cliente y el servidor. Su función principal es permitir el intercambio de información mediante protocolos como WebSockets o API REST, lo que facilita la transmisión de datos en tiempo real, asegurando que las solicitudes y respuestas fluyan de manera rápida y eficiente.

Servidor y base de datos

El servidor de aplicaciones es el núcleo que procesa las solicitudes enviadas por usuarios y técnicos. La base de datos es la encargada de almacenar información crítica, como credenciales de usuarios, tickets de soporte, estados de los casos y eventos del sistema. A través de consultas SQL, se accede a la información necesaria para validar usuarios, registrar actividades y dar seguimiento a los incidentes.

Módulo de soporte técnico

Este módulo corresponde al área operativa de los técnicos de soporte. A través de un panel técnico (API REST / Web), los especialistas reciben, gestionan y solucionan las solicitudes de los usuarios. Asimismo, cuenta con un registro de actividades que almacena un historial detallado de las acciones realizadas, lo cual facilita la trazabilidad del servicio.

Notificación de alertas

Una de las características más relevantes del sistema es el componente de notificación de alertas. Este se encarga de enviar mensajes push y notificaciones en tiempo real, alertando al técnico sobre casos críticos o urgentes. Con ello se asegura una respuesta rápida y oportuna a las incidencias de mayor importancia.

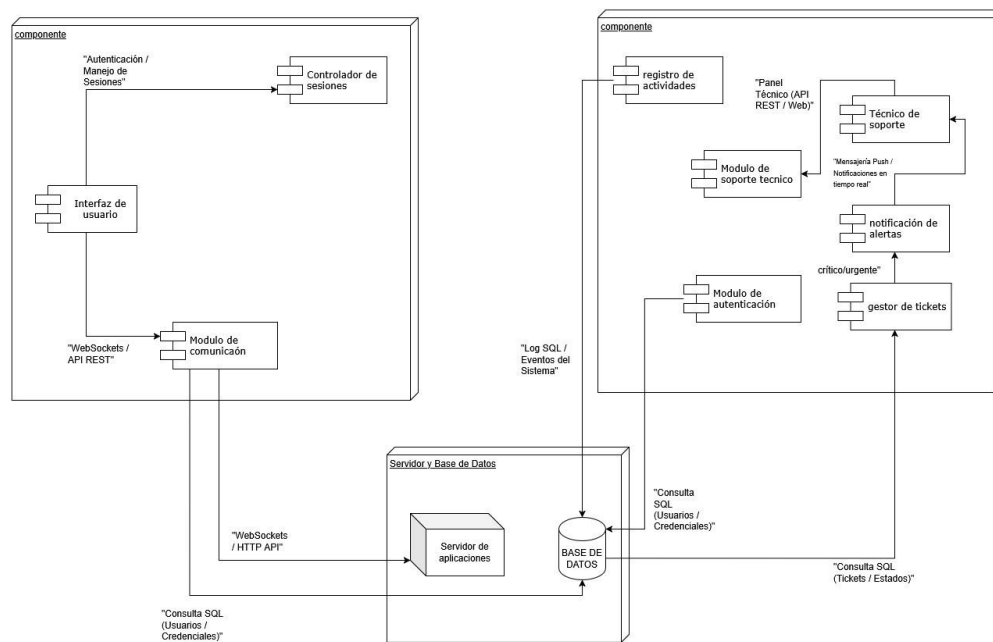
Gestor de tickets

El gestor de tickets organiza y administra todos los reportes de soporte. Permite clasificar los casos en función de su estado (abierto, en proceso o cerrado), facilitando el control y seguimiento de cada incidencia. Este módulo se encuentra directamente vinculado con la base de datos, de manera que las actualizaciones de estado se registran de forma automática y centralizada.

Módulo de autenticación

Para garantizar la seguridad del sistema, se cuenta con un módulo de autenticación que valida las credenciales tanto de usuarios como de técnicos. Este componente asegura que solo las personas autorizadas puedan acceder a la plataforma, protegiendo la integridad y confidencialidad de la información.

Diagrama de componentes



Identificación de capas de los componentes

Capa / Componente	Por hacer	En proceso	Hecho
Presentación (Frontend)	Mejorar diseño responsivo. Agregar accesibilidad (contraste, etiquetas).	Pantalla de inicio de sesión (usuario y contraseña).	Interfaz básica diseñada (logo, colores, formulario de login).
Lógica de Negocio (Backend)	Crear validación de usuarios. Definir roles (admin, cliente).	Implementación de autenticación (login).	Estructura inicial del flujo de login.
Datos (Base de datos)	Diseñar modelo de base de datos. Configurar almacenamiento seguro de contraseñas.	Integración con la lógica de login.	Creada base de datos de prueba para usuarios.
Seguridad	Integrar cifrado de contraseñas (hash + salt).	Evaluar políticas de bloqueo por intentos y auditoría de accesos.	Política inicial de usuario y contraseña en login (ya implementada).
Sesión Iniciada <i>(UI + Backend)</i>	Mejorar mensaje de bienvenida; registrar logs de sesión; implementar expiración automática (timeout).	Integrar auditoría/registro de sesiones con la base de datos; métricas de uso.	Pantalla de bienvenida: “¡Bienvenido a Support Bot Remote!”; botón Cerrar sesión funcional; confirmación de inicio de sesión y redirección según rol.

Capa de inicio de sesión Nexa Support



The login form is centered on a light gray background with a subtle circuit pattern. It features a white rounded rectangle with a light blue gradient header. The title 'Inicio de Sesión' is in bold black text. Below it, the 'Usuario' label is followed by a white input field with a light blue border and the placeholder 'Ingresa su usuario'. The 'Contraseña' label is followed by a similar input field with the placeholder 'Ingresa su contraseña'. A blue link 'Mostrar contraseña' is positioned below the password field. A solid blue button with the text 'Ingresar' is centered below the inputs. At the bottom, two blue links are displayed: '¿No eres cliente aún? Regístrate' and '¿Olvidaste tu contraseña?'.

Inicio de Sesión

Usuario

Contraseña

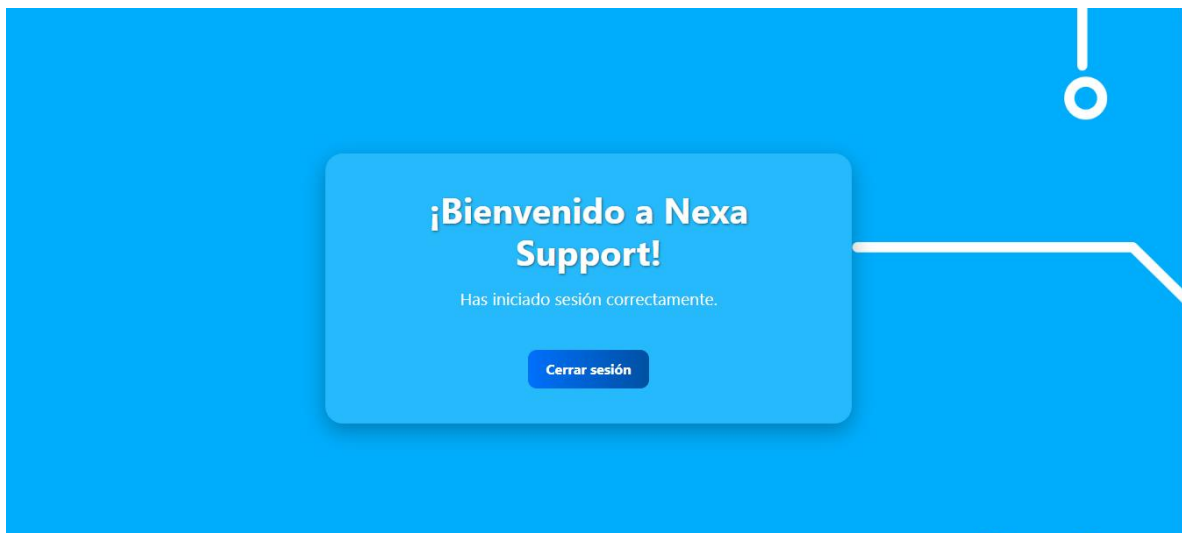
[Mostrar contraseña](#)

Ingresar

[¿No eres cliente aún? Regístrate](#)

[¿Olvidaste tu contraseña?](#)

Capa de sesión iniciada



Metodología Kanban

Para nuestro proyecto de gestión de soporte técnico remoto, hemos considerado varias metodologías de desarrollo de software. Dado el tamaño reducido de nuestro equipo (3 personas) y el presupuesto limitado, creo que la metodología Kanban es la más adecuada. A continuación, presento una introducción a Kanban y sus beneficios específicos para nuestro contexto.

¿Qué es Kanban?

Kanban es una metodología ágil y visual que facilita la gestión del flujo de trabajo y la entrega continua de tareas. Originada en el sistema de producción de Toyota, Kanban ha sido adaptada para el desarrollo de software, permitiendo visualizar el trabajo, limitar las tareas en progreso y mejorar continuamente los procesos.

Beneficios de Kanban para Nuestro Proyecto

Simplicidad y Flexibilidad:

Kanban es fácil de entender y aplicar. No requiere roles específicos ni ceremonias formales, lo que lo hace ideal para nuestro equipo pequeño.

Nos permite adaptarnos rápidamente a cambios en los requisitos del proyecto sin necesidad de una replanificación exhaustiva.

Visualización del Trabajo:

Utilizando un tablero Kanban, podemos ver todas las tareas en diferentes estados (Por hacer, En progreso, Hecho), lo que mejora la transparencia y la comunicación dentro del equipo.

Esta visualización nos ayudará a identificar cuellos de botella y áreas de mejora.

Enfoque en la Entrega Continua:

Kanban permite entregar pequeñas partes del proyecto de manera continua, facilitando la obtención de feedback temprano y frecuente.

Esto es crucial en un entorno académico, donde es importante demostrar progreso regularmente.

Reducción del Trabajo en Progreso (WIP):

Limitar el trabajo en progreso nos ayudará a mantenernos enfocados y a evitar la sobrecarga, mejorando así nuestra eficiencia y productividad.

Implementación de Kanban

Creación del Tablero Kanban:

Utilizar una herramienta como Trello o Notion, esta última bastante cómoda debido a sus flexibilidad y mi experiencia de uso, para crear nuestro tablero Kanban, dividiéndolo en columnas: "Por hacer", "En progreso", y "Hecho".

Definición de Tareas:

Desglosar el proyecto en tareas pequeñas y manejables, asegurándonos de que cada tarea sea clara y específica.

Límite del Trabajo en Progreso (WIP):

Establecer un límite en la cantidad de tareas que pueden estar en progreso al mismo tiempo para mantener el enfoque y la eficiencia.

Revisión y Adaptación Regular:

Revisar el progreso del tablero regularmente, ajustando prioridades y adaptando nuestro flujo de trabajo según sea necesario para mejorar continuamente.

Adoptar la metodología Kanban permitirá gestionar nuestro proyecto de manera eficiente, asegurando que podamos adaptarnos a los cambios y mantener un progreso constante. Con su enfoque en la visualización del trabajo y la entrega continua, Kanban es la opción ideal para nuestro equipo pequeño y nuestro entorno académico.

Mecanismos de seguridad en usuarios y contraseñas

Para garantizar la seguridad en el acceso al aplicativo *Nexa Support*, se implementan los siguientes mecanismos:

1. Almacenamiento seguro de contraseñas

- Las contraseñas no se guardan en texto plano.
- Se emplea un algoritmo de hash seguro como bcrypt o Argon2, que transforma la contraseña en una cadena irreconocible.
- Incluso si la base de datos se ve comprometida, los atacantes no podrán obtener las contraseñas reales.

2. Validación en el inicio de sesión

- El sistema solicita usuario y contraseña.
- La contraseña ingresada se cifra y se compara con la almacenada en la base de datos.
- Solo si coincide el hash, se permite el acceso.

3. Política de contraseñas seguras

- Longitud mínima de 8 a 12 caracteres.
- Inclusión de letras mayúsculas, minúsculas, números y símbolos.
- Evitar contraseñas comunes como “123456” o “password”.

4. Protección contra ataques de fuerza bruta

Se limita el número de intentos fallidos de inicio de sesión.

Si un usuario excede el límite, su cuenta se bloquea temporalmente o se activa un captcha.

5. Recuperación de contraseña segura

- El enlace de recuperación se envía únicamente al correo registrado.
- Este enlace tiene una vigencia limitada (ejemplo: 15 minutos).

El aplicativo *Support Bot Remote* asegura la confidencialidad y protección de sus usuarios implementando almacenamiento cifrado de contraseñas, políticas de contraseñas seguras, validaciones en el login y medidas contra ataques de fuerza bruta.



Nexa Support
Soporte remoto eficiente y confiable

Modo Oscuro

Inicio de Sesión

Usuario
admin

Contraseña
....

Mostrar contraseña

Ingresar

¿No eres cliente aún? Regístrate
¿Olvidaste tu contraseña?

© 2025 Nexa Support. Todos los derechos reservados.

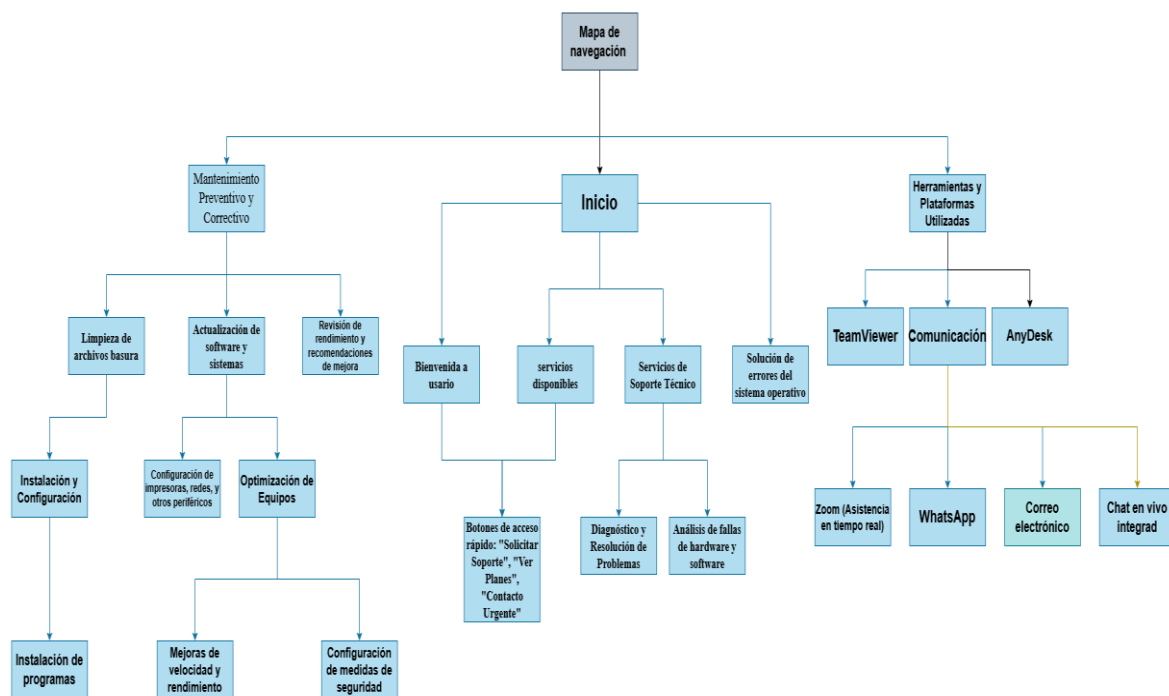
Mapa de navegación

El mapa de navegación diseñado corresponde a una estructura jerárquica, ya que organiza la información de manera estructurada, partiendo de un nodo principal ("Inicio") que se ramifica en diferentes secciones temáticas relevantes al soporte técnico remoto, tales como: Mantenimiento Preventivo y Correctivo, Servicios de Soporte Técnico, Solución de errores del sistema operativo, y Herramientas y Plataformas Utilizadas.

Cada una de estas secciones se divide a su vez en subsecciones específicas, permitiendo un acceso claro, ordenado y lógico a los diferentes contenidos. Este tipo de navegación facilita la comprensión del usuario, ya que sigue un esquema descendente donde la información va de lo general a lo particular, priorizando la usabilidad y la experiencia del usuario.

La elección de un mapa jerárquico responde a la necesidad de mantener un flujo de navegación claro y eficiente, garantizando que el usuario pueda encontrar fácilmente los servicios o recursos que necesita dentro del sitio web de soporte técnico remoto.

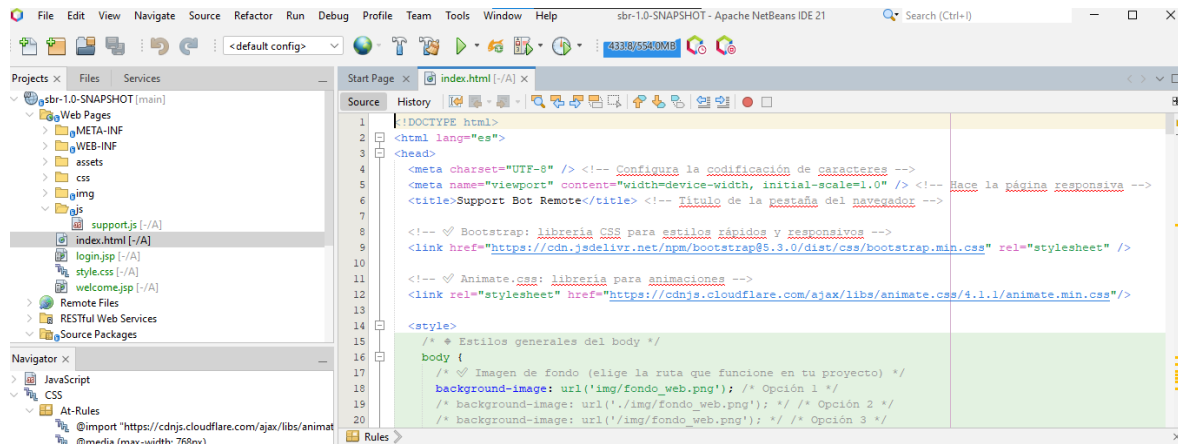
Mapa de navegación jerárquico



Codificar cada módulo en el lenguaje seleccionado.

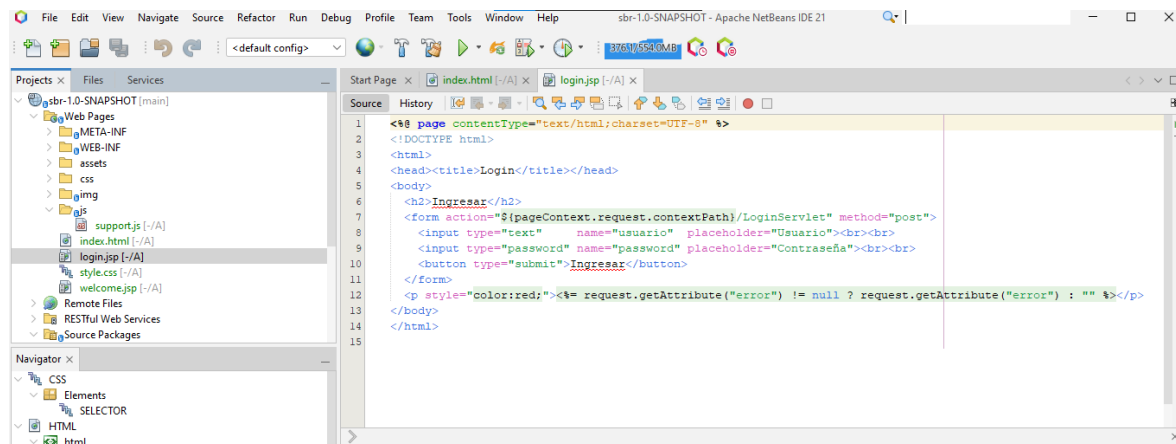
index.html

Es la página de inicio del sistema. Contiene el código HTML base con estilos, referencias a librerías (Bootstrap y Animate.css), y define el diseño inicial que ve el usuario cuando entra.



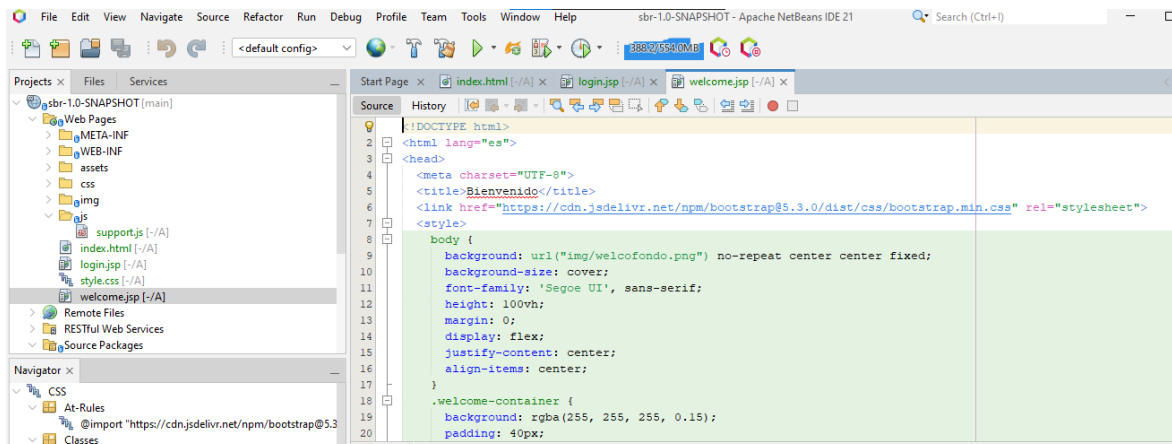
login.jsp

Página que contiene el formulario de inicio de sesión (usuario y contraseña). Envía los datos al backend para validarlos.



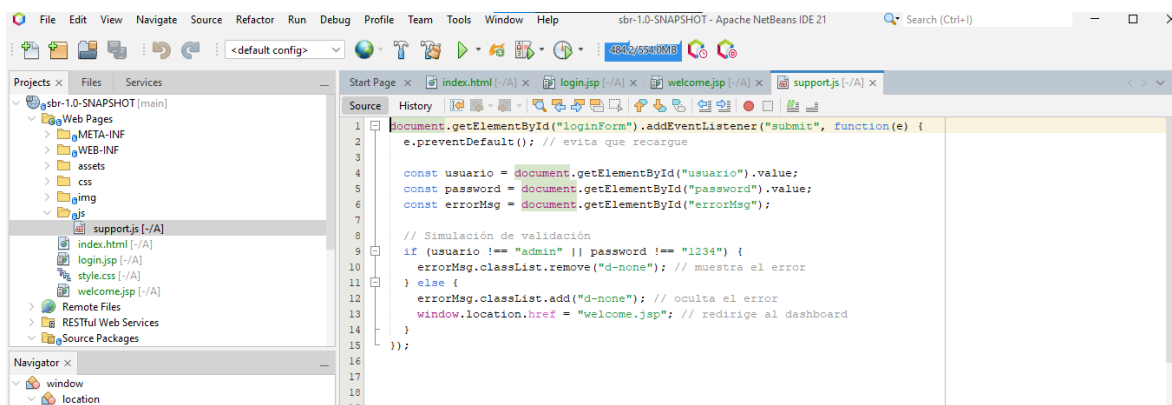
welcome.jsp

Página a la que se redirige al usuario después de iniciar sesión correctamente. Sirve como pantalla de bienvenida o panel principal.



support.js

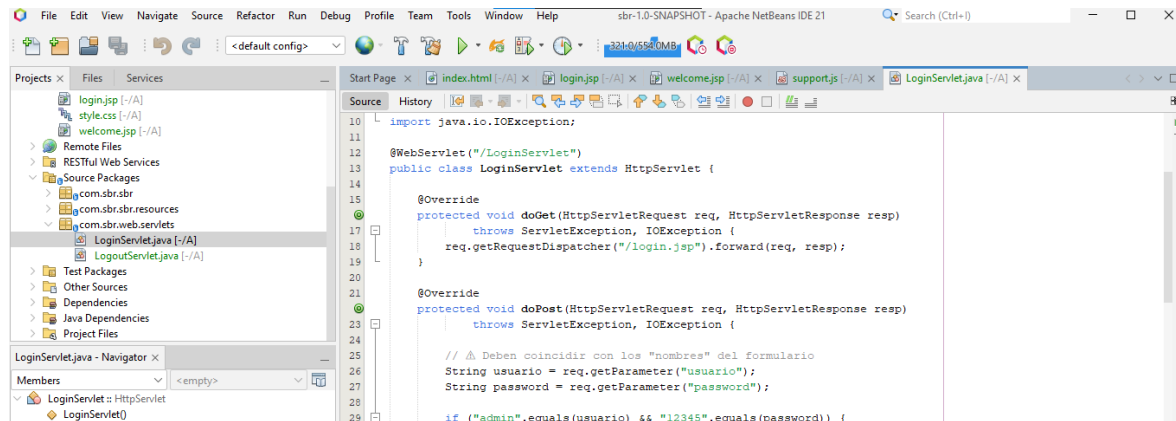
Archivo con funciones en JavaScript para agregar interactividad (validaciones del formulario, animaciones, mensajes dinámicos, etc.).



LoginServlet.java

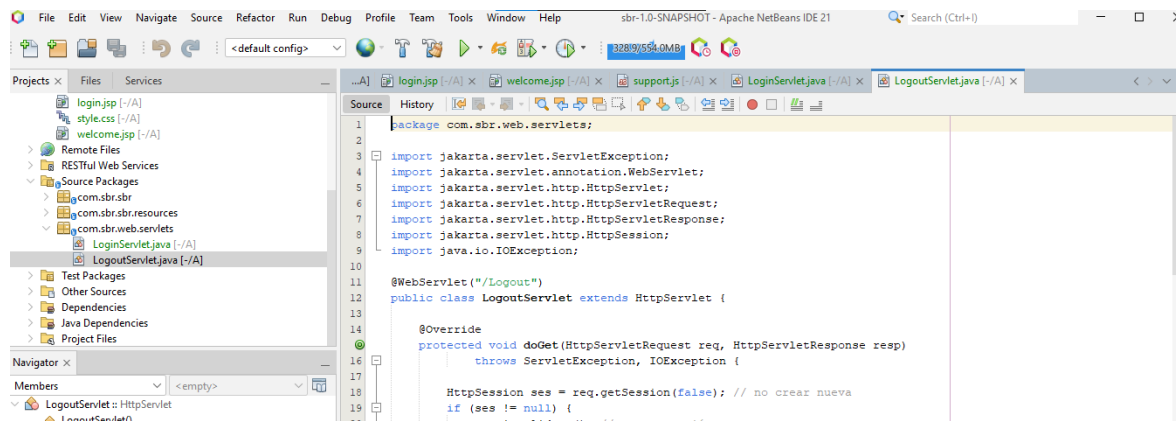
Se encarga de recibir los datos del formulario de login (usuario y contraseña). Valida la información comparándola con la base de datos.

- Si los datos son correctos → abre la sesión del usuario.
- Si son incorrectos → devuelve un mensaje de error.

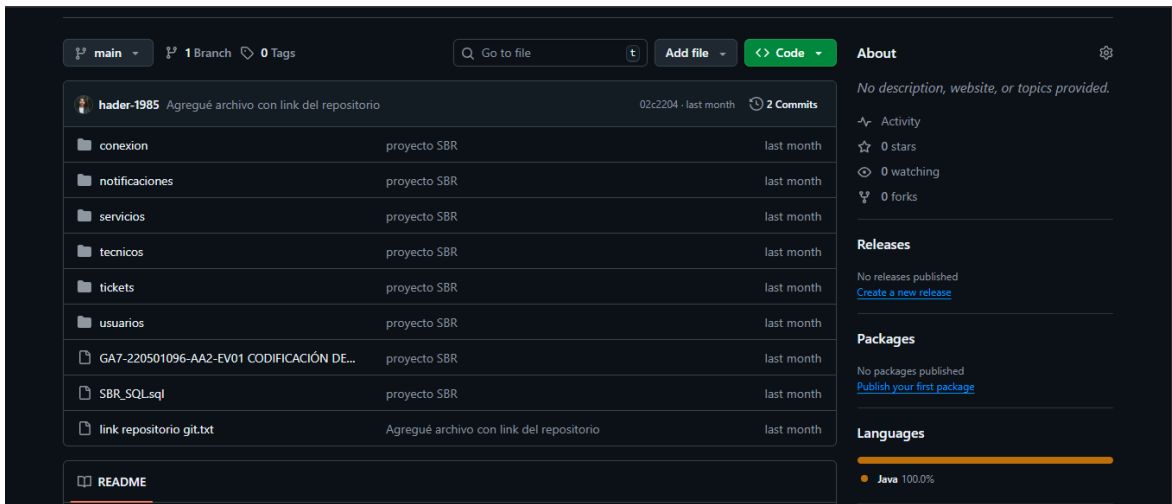


LogoutServlet.java

Finaliza la sesión activa del usuario y lo redirige a la página de inicio (index.html o login.jsp).



Manejar un repositorio de control de versiones (GIT, SVN, Otros)



El uso de un repositorio de control de versiones, como Git, es fundamental en proyectos de desarrollo de software porque permite llevar un registro detallado de todos los cambios realizados en el código y en los archivos asociados. Gracias a este sistema, cada modificación queda documentada con fecha, autor y descripción, lo que facilita identificar errores, recuperar versiones anteriores y mantener un historial organizado del proyecto.

Además, el repositorio centralizado en plataformas como GitHub brinda la posibilidad de trabajar en equipo de manera eficiente. Cada integrante puede aportar mejoras o correcciones sin riesgo de sobrescribir el trabajo de otros, ya que Git administra las versiones y resuelve posibles conflictos. Esto asegura que todos los participantes tengan acceso a la misma versión actualizada del proyecto, evitando confusiones o pérdidas de información.

En el caso del proyecto, subir los archivos de código y las bases de datos al repositorio no solo ofrece seguridad y respaldo, sino también transparencia y colaboración. De esta manera, el repositorio se convierte en una herramienta indispensable para garantizar la continuidad, la organización y el éxito del desarrollo del sistema.

Determinar las librerías necesarias en cada capa de la aplicación.

1. Capa de presentación (Front-End / Vista)

- Es la que interactúa con el usuario.
- Archivos: index.html, login.jsp, style.css, support.js, welcome.jsp.
- Librerías necesarias:

- JSP/Servlet API (para que las páginas JSP funcionen).
- Bootstrap o CSS frameworks (si lo usas).
- Librerías de JavaScript (como jQuery, si decides incluirlo).

2. Capa de lógica de negocio (Servicios / Controladores)

- Contiene la lógica que controla el flujo entre la vista y la base de datos.
- Archivos: LoginServlet.java, LogoutServlet.java, clases dentro de servicios, tecnicos, tickets.

Librerías necesarias:

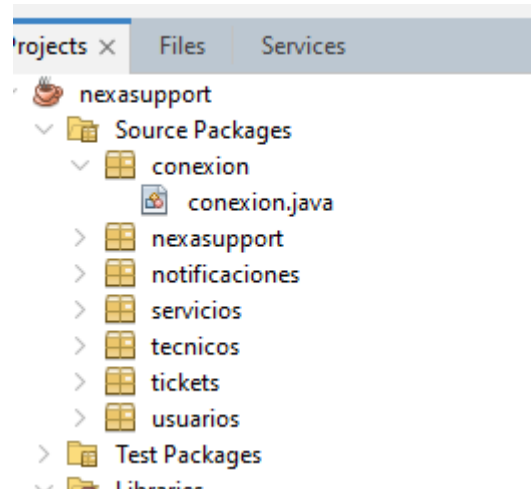
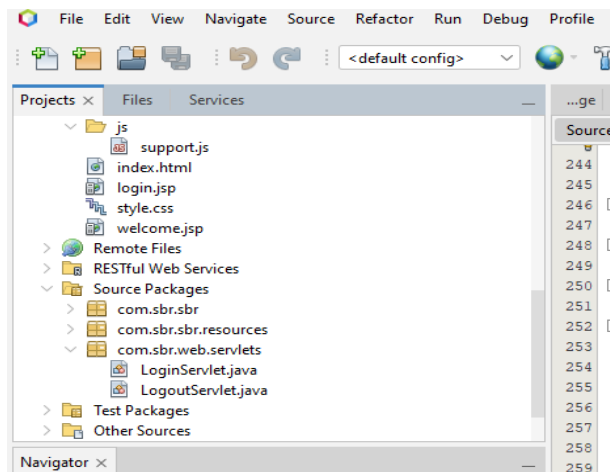
- Servlet API (para manejar las peticiones HTTP).
- Java Standard (JDK) para toda la lógica interna.
- Posiblemente librerías externas si manejas validaciones, JSON, o seguridad (por ejemplo Gson para JSON).

3. Capa de acceso a datos (DAO / Persistencia)

- Encargada de comunicarse con la base de datos.
- Archivos: conexion.java y las clases relacionadas a usuarios, técnicos, tickets, etc.
- Librerías necesarias:
 - MySQL Connector/J → la que ya tienes (mysql-connector-j-8.4.0.jar) para poder conectarte a MySQL.
 - Si usas frameworks ORM como Hibernate o JPA, necesitarías sus librerías (aunque no parece ser tu caso aún).

4. Capa de utilidades (opcional)

- Métodos de ayuda o funciones que se usan en varias capas.
- Librerías necesarias: dependen de lo que necesites (ejemplo: manejo de fechas, logs, encriptación, etc.).



Determinar los frameworks en cada capa de la aplicación.

Un **framework** es un conjunto de herramientas y librerías que te ayudan a programar más rápido y ordenado, siguiendo buenas prácticas. En tu caso, como tu proyecto es **Java Web con JSP/Servlets y MySQL**, los frameworks más comunes serían estos:

Capas y frameworks en Nexa Support

Capa de Presentación (Vista / Front-End)

- Archivos: index.html, login.jsp, welcome.jsp, style.css, support.js.
- Frameworks posibles:
- **JSP (JavaServer Pages)** → para generar las páginas dinámicas.
- **Bootstrap** (si lo agregas para estilos CSS).
- **jQuery o JavaScript Vanilla** (dependiendo si usas librerías JS externas).

2. Capa de Lógica de Negocio (Servicios / Controladores)

- Archivos: LoginServlet.java, LogoutServlet.java, clases de servicios, tecnicos, tickets.
- Frameworks posibles:
- **Servlet API (Jakarta/Javax)** → base para manejar peticiones y respuestas HTTP.
- (Opcional) **Spring Framework o Jakarta EE** → si quieres ampliar la arquitectura y hacerla más profesional, aunque en tus capturas parece que estás usando lo básico (Servlets).

3. Capa de Acceso a Datos (Persistencia / DAO)

- Archivos: `conexion.java`, clases de usuarios, tecnicos, tickets que interactúan con la BD.
- Frameworks posibles:
- **JDBC (Java Database Connectivity)** → ya lo usas con `mysql-connector`.
- (Opcional) **Hibernate / JPA** → frameworks ORM que automatizan las consultas SQL (si decides implementar más adelante).

4. Capa de Utilidades (opcional)

- Métodos generales de ayuda (validaciones, seguridad, logs, etc.).
- Frameworks posibles:
- **Log4j o SLF4J** (para manejo de logs).
- **Apache Commons** (para utilidades generales).

Se recomienda dividir el módulo a desarrollar en componentes que pueden ser reutilizados.

En el desarrollo de mi proyecto se recomienda dividir el módulo en componentes reutilizables, ya que esto permite organizar mejor el sistema y aprovechar el código en diferentes partes de la aplicación.

Por ejemplo, en mi caso tengo un componente de **conexión a la base de datos** (`conexion.java`) que utilizo en los diferentes módulos como *usuarios*, *técnicos* y *tickets*. De esta manera no tengo que volver a programar la conexión en cada parte, sino que con un solo componente puedo reutilizarlo varias veces.

También tengo componentes específicos como el **LoginServlet**, que se encarga únicamente del inicio de sesión, y el **LogoutServlet**, que maneja el cierre de sesión. Al tener cada funcionalidad en un componente separado, se facilita el mantenimiento, ya que si debo hacer un cambio, solo lo hago una vez y se aplica en todo el proyecto.

Además, esta división en componentes ayuda a que el sistema sea más escalable y a que se pueda trabajar en equipo sin que el código se vuelva desordenado. En conclusión, organizar el proyecto en módulos y componentes reutilizables hace que el software sea más claro, mantenible y eficiente.



Aplicar la codificación siguiendo las buenas prácticas de escritura de código.

```
<style>
/* ✦ Estilos generales del body */
body {
    /* ♥ Imagen de fondo (elige la ruta que funcione en tu proyecto) */
    background-image: url('img/fondo_web.png'); /* Opción 1 */
    /* background-image: url('./img/fondo_web.png'); */ /* Opción 2 */
    /* background-image: url('/img/fondo_web.png'); */ /* Opción 3 */
    /* background-image: url('/sbr-1.0-SNAPSHOT/img/fondo_web.png'); */ /* Opción 4 (Tomcat) */

    background-size: cover; /* La imagen cubre toda la pantalla */
    background-repeat: no-repeat; /* Evita que se repita */
    background-position: center; /* Centrada */
    margin: 0;
    min-height: 100vh; /* Ocupa el alto completo */
    display: flex;
    flex-direction: column;
    transition: background 0.3s ease, color 0.3s ease; /* Transiciones suaves */
}

```

Uso de comentarios en el código

- Estás explicando cada línea y cada opción (/* Imagen de fondo (elige la ruta que funcione en tu proyecto) */, /* Centrada */, etc.).
- Esto facilita que cualquier programador (o incluso tú mismo en el futuro) entienda rápidamente para qué sirve cada instrucción sin necesidad de adivinar.

Organización y claridad

- Separas las propiedades por bloques (background, margin, display) con espacios y tabulación consistente.
- Eso hace que el CSS sea más legible y fácil de mantener.

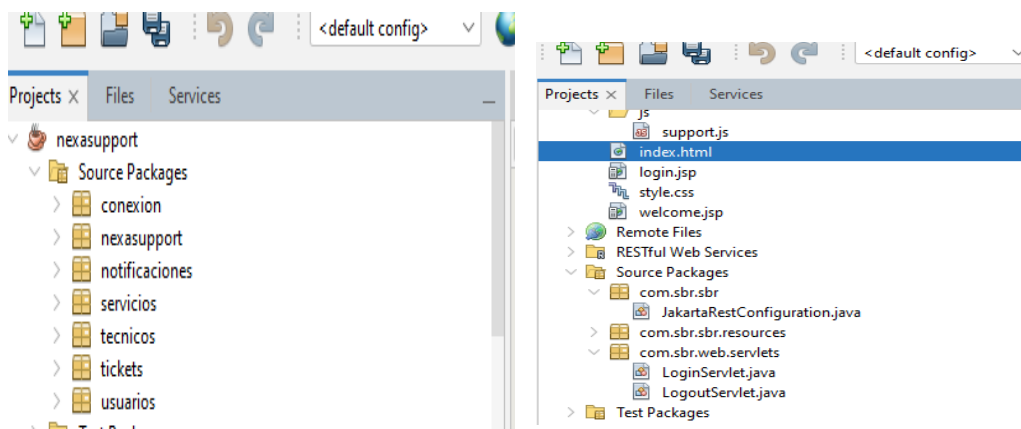
Opciones alternativas comentadas

- Has dejado varias formas de cargar la imagen de fondo (Opción 1, 2, 3, 4) pero las mantienes comentadas.
- Esta práctica evita borrar código que puede ser útil después y permite hacer cambios rápidos solo descomentando.

Buenas prácticas en CSS moderno

- Usar background-size: cover; para que la imagen siempre ocupe toda la pantalla.
- transition para dar un efecto suave y mejorar la experiencia de usuario.
- display: flex; con flex-direction: column; que prepara la estructura de la página con diseño adaptable.

Se recomienda dividir el código fuente de cada componente que corresponde a un módulo en paquetes con nombres de fácil entendimiento.



División del código fuente en paquetes con nombres entendibles

En mi proyecto he aplicado la recomendación de organizar el código en paquetes con nombres descriptivos, lo que facilita la comprensión, el mantenimiento y la escalabilidad de la aplicación.

- Por ejemplo, el paquete `conexion` contiene la clase encargada de establecer la conexión con la base de datos.
- Los paquetes `usuarios`, `tecnicos`, `tickets`, `notificaciones` y `servicios` permiten separar la lógica según cada módulo de la aplicación, de manera que cada uno gestiona sus propias funcionalidades.
- En la parte web, también mantengo organizados los archivos en carpetas claras como `js` para los scripts, `style.css` para los estilos, y páginas como `index.html`, `login.jsp` y `welcome.jsp`, que forman la interfaz de usuario.
- Además, se incluyen paquetes como `com.nexa.web.servlets` para agrupar los servlets y manejar el flujo de la aplicación de manera estructurada.

Esta forma de organización sigue las buenas prácticas de modularidad, ya que hace que el código sea más fácil de leer, depurar y reutilizar, además de permitir que otros desarrolladores comprendan rápidamente la función de cada parte del sistema.

Aplicar los patrones de diseño de acuerdo con la arquitectura del software por componente

En el desarrollo de mi proyecto he aplicado algunos **patrones de diseño** que facilitan la organización y el mantenimiento del sistema. Estos patrones permiten que la arquitectura por componentes sea más clara y escalable.

1. Patrón DAO (Data Access Object)

- Lo aplico en la capa de conexión y en los paquetes como `usuarios`, `tecnicos`, `tickets` y `servicios`.
- Este patrón permite separar la lógica de negocio del acceso a los datos, de manera que las clases DAO se encargan exclusivamente de interactuar con la base de datos (consultar, insertar, actualizar o eliminar).
- Con esto se logra que el código sea más reutilizable y fácil de mantener.

2. Patrón MVC (Modelo – Vista – Controlador)

- Lo aplico en la parte web del sistema.
- El **modelo** está representado por las clases Java que manejan los datos (`usuarios`, `tecnicos`, etc.).
- La **vista** son los archivos como `index.html`, `login.jsp`, `welcome.jsp` y las hojas de estilo `style.css`.
- El **controlador** lo representan los servlets (`LoginServlet.java`, `LogoutServlet.java`) y las funciones en `support.js` que controlan la interacción entre el usuario y el sistema.

- Este patrón facilita que la interfaz de usuario esté separada de la lógica del negocio.

3. Patrón Singleton (Conexión a base de datos)

- Se utiliza en el paquete conexion.
- Garantiza que la conexión a la base de datos se cree una sola vez y se reutilice en toda la aplicación, evitando problemas de múltiples conexiones abiertas.

Gracias a estos patrones, la aplicación queda dividida en **componentes claros y reutilizables**, lo que permite trabajar en equipo sin generar conflictos y asegura que el sistema pueda crecer sin perder orden.

Aplicación de patrones de diseño en la arquitectura de software por componentes.

En el desarrollo de aplicaciones, es importante organizar el código de manera estructurada para que sea más fácil de mantener, reutilizar y escalar en el futuro. Para lograrlo, se aplican patrones de diseño que son soluciones ya probadas a problemas comunes en la programación.

1. ¿Qué son los patrones de diseño?

Los patrones de diseño son guías que nos indican cómo organizar las partes de una aplicación para resolver problemas de manera eficiente y ordenada. Algunos patrones comunes son:

- MVC (Modelo – Vista – Controlador): separa la interfaz del usuario, la lógica de negocio y el acceso a datos.
- DAO (Data Access Object): crea clases específicas para acceder a la base de datos.
- Singleton: garantiza que una clase solo tenga una única instancia.
- Factory: centraliza la creación de objetos.

2. ¿Qué es la arquitectura de software por componentes?

La arquitectura por componentes organiza el sistema en módulos o piezas independientes, donde cada componente tiene una responsabilidad clara. Esto evita que todo quede mezclado en un solo archivo.

Ejemplo de componentes en una aplicación web:

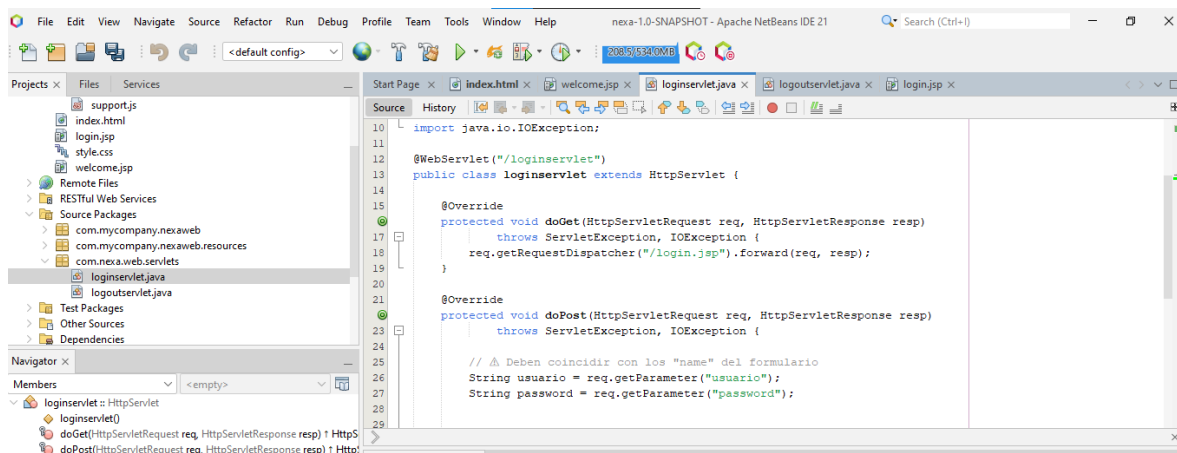
- Componente de presentación (Vista): maneja la interfaz de usuario (HTML, CSS, JSP).
- Componente de lógica de negocio (Controlador): gestiona las operaciones principales de la aplicación (Servlets en Java).
- Componente de acceso a datos (Modelo): interactúa con la base de datos (clases DAO).

3. Aplicación de patrones en la arquitectura por componentes

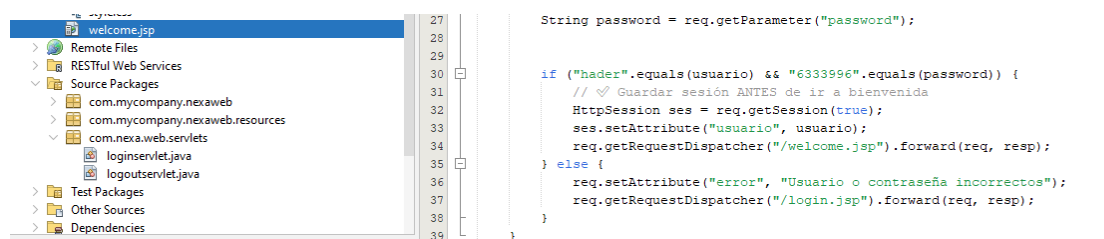
Cuando se dice “Aplicar los patrones de diseño de acuerdo con la arquitectura del software por componente”, significa que debemos organizar el sistema usando patrones para que cada componente cumpla su función correctamente:

- Vista (View): formulario index.html o login.jsp donde el usuario ingresa datos.
- Controlador (Controller): LoginServlet.java recibe los datos enviados por el formulario, los procesa y decide qué hacer.
- Modelo (Model): clases como Usuario.java y UsuarioDAO.java que representan los usuarios y se encargan de consultar o registrar información en la base de datos.

4. Ejemplo aplicado (Login con Servlets)



Modelo – Acceso a datos (UsuarioDAO.java)



Se deben realizar las pruebas unitarias de cada módulo.

Explicación del Testeo en Postman – Proyecto Nexa Support

Para garantizar que el servicio web de **registro e inicio de sesión** funciona correctamente en el proyecto *Nexa Support*, realicé pruebas utilizando **Postman**, una herramienta muy útil para verificar y consumir APIs o servicios web.

1. Configuración de la petición

En Postman configuré una petición de tipo **GET** hacia la ruta:

<http://localhost:8080/nexa/index.html>

Esta URL corresponde al despliegue local del proyecto dentro del servidor Tomcat, lo que me permitió comprobar si la aplicación respondía correctamente desde el navegador y también desde Postman.

2. Respuesta del servidor

Al ejecutar la petición, el servidor devolvió un **Status 200 OK**, lo cual indica que la página y el servicio estaban activos y funcionando.

- En la pestaña **Raw** de Postman se puede ver todo el contenido HTML del proyecto (estructura de Nexa Support, referencias a Bootstrap, Animate.css, etc.).
- En la pestaña **Preview** se observa de manera visual la interfaz, mostrando el formulario de **Inicio de Sesión** junto con el título del sistema y el modo oscuro habilitado.

3. Validación del inicio de sesión

El objetivo de esta prueba era comprobar que el servicio web recibe correctamente las solicitudes y devuelve las respuestas adecuadas:

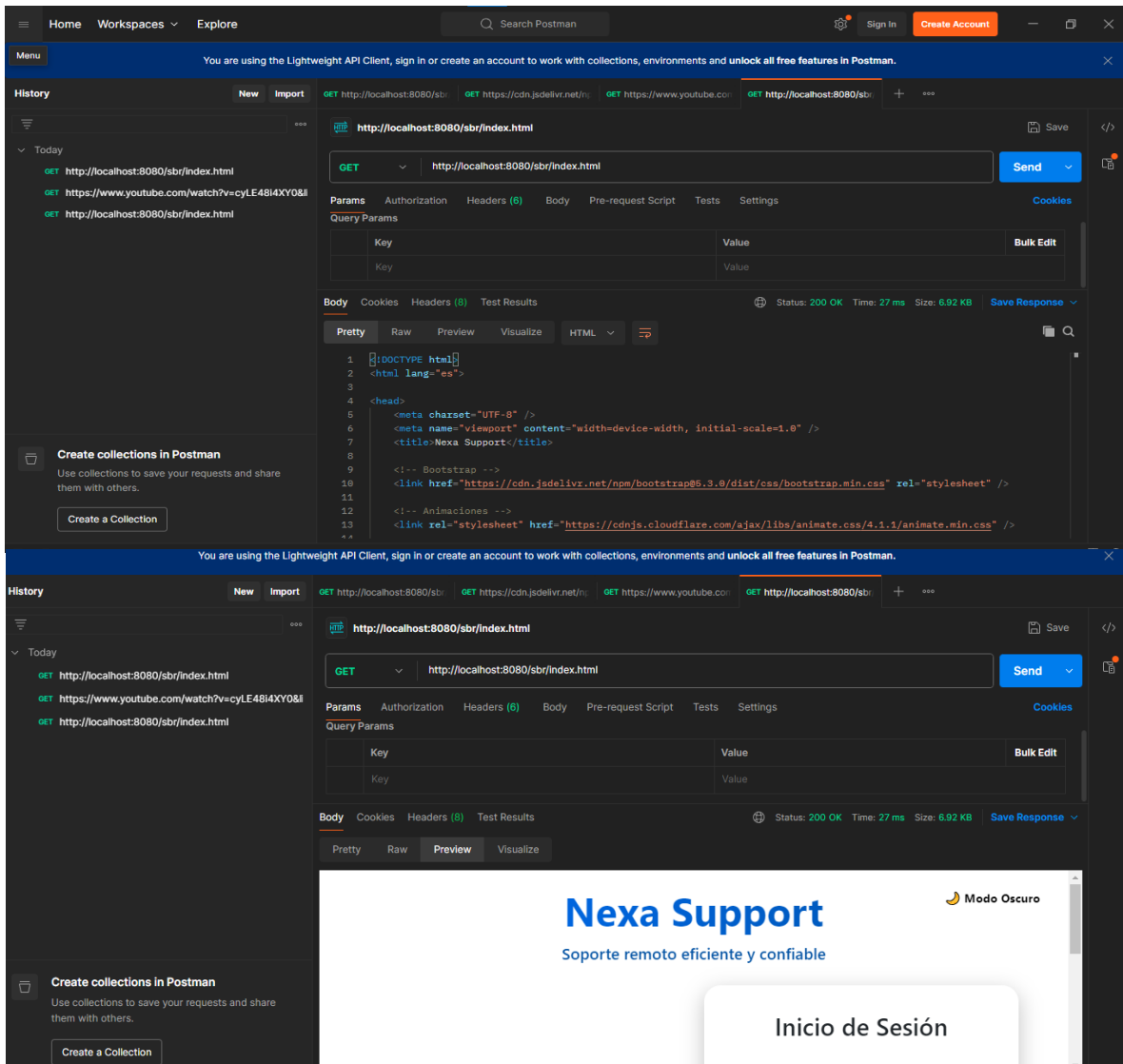
- Si el usuario y la contraseña son correctos, debe mostrar un mensaje de **autenticación satisfactoria**.

- En caso contrario, el servicio debe devolver un error de autenticación.

4. Importancia del testeo

El uso de Postman en este caso asegura que:

- El servicio web funciona de forma independiente al navegador.
- Se valida que la ruta está bien configurada en el servidor local.
- Se puede realizar un control de calidad rápido antes de implementar funciones adicionales como el registro de usuarios o la conexión a base de datos.



Configuraciones de servidores y de bases de datos.

Tabla usuarios

Explicación técnica: esta tabla funciona como núcleo de la base de datos, ya que a partir de aquí se enlazan otras entidades como técnicos, notificaciones y tickets.

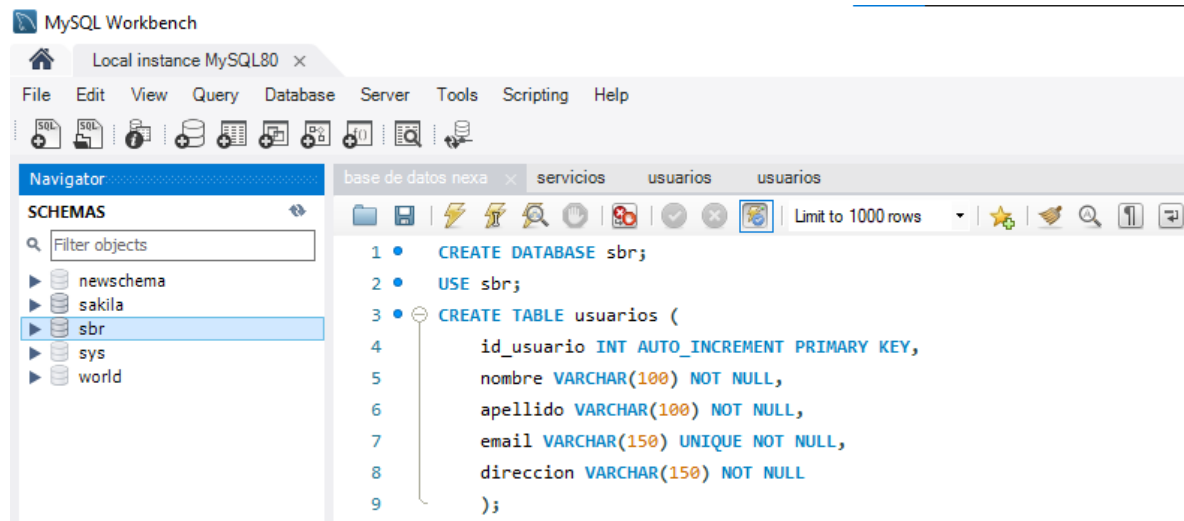


Tabla técnico

Explicación técnica: se usa id_usuario como clave foránea, lo que significa que un técnico es a la vez un usuario. De esta manera no se duplican datos, y se aprovecha la información del usuario original.

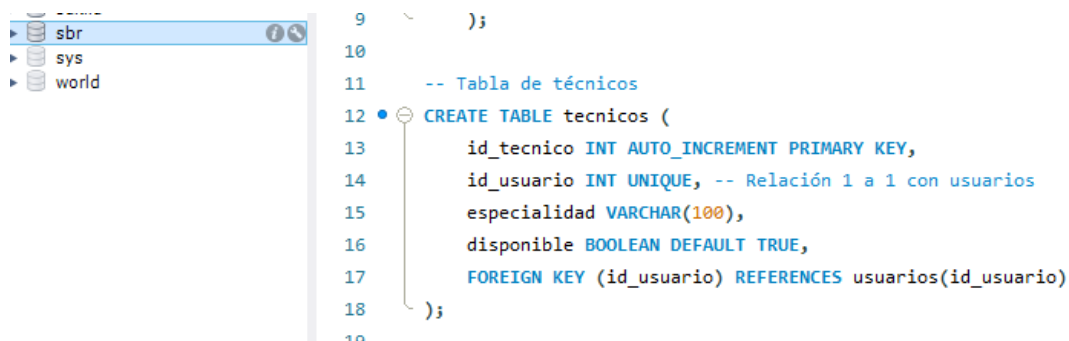


Tabla de servicios

Explicación técnica: esta tabla funciona como un catálogo de servicios disponibles en el sistema.

```
--      ''
19
20      -- Tabla de servicios
21      CREATE TABLE servicios (
22          id_servicio INT AUTO_INCREMENT PRIMARY KEY,
23          nombre_servicio VARCHAR(100) NOT NULL,
24          descripcion TEXT,
25          precio DECIMAL(10,2) DEFAULT 0.00
26      );
27
```

Tabla de tickets de soporte

Explicación técnica: es la tabla central del sistema porque conecta cliente (usuario), servicio solicitado y técnico asignado. Permite llevar el control de cada caso de soporte.

```
26      );
27
28      -- Tabla de tickets de soporte
29      CREATE TABLE tickets (
30          id_ticket INT AUTO_INCREMENT PRIMARY KEY,
31          id_usuario INT,      -- Cliente que abre el ticket
32          id_servicio INT,     -- Servicio solicitado
33          id_tecnico INT,      -- Técnico asignado
34          descripcion TEXT,
35          estado ENUM('pendiente', 'en_proceso', 'resuelto') DEFAULT 'pendiente',
36          fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
37          FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario),
38          FOREIGN KEY (id_servicio) REFERENCES servicios(id_servicio),
39          FOREIGN KEY (id_tecnico) REFERENCES tecnicos(id_tecnico)
40      );
```

Explicación técnica: permite mantener informado al usuario sobre el estado de su ticket o cualquier cambio en el sistema.

```
--      ''
41
42      -- Tabla de notificaciones
43      CREATE TABLE notificaciones (
44          id_notificacion INT AUTO_INCREMENT PRIMARY KEY,
45          id_usuario INT,
46          mensaje VARCHAR(255),
47          leido BOOLEAN DEFAULT FALSE,
48          fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
49          FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
50      );
```

Schema: sbr

Documentar los ambientes de desarrollo y pruebas.

Dejo como evidencia video de desarrollo y pruebas dentro de la carpeta.



Documentar los
ambientes de
desarrollo y
pruebas.

Conclusiones

El desarrollo del proyecto **Nexa Support** permitió comprobar la importancia de integrar herramientas modernas en la creación de sistemas de soporte técnico remoto. Gracias a la combinación de **Apache NetBeans** como entorno de desarrollo y **MySQL** como gestor de base de datos, se logró implementar una solución que asegura la correcta administración de usuarios, técnicos, servicios y tickets, optimizando el flujo de trabajo y fortaleciendo la trazabilidad de las incidencias.

Asimismo, se demostró que el diseño modular y la aplicación de principios de seguridad son fundamentales para garantizar la escalabilidad y confiabilidad del sistema. La inclusión de notificaciones automáticas y estados de ticket aporta valor agregado al proceso de atención, promoviendo la transparencia y mejorando la experiencia del usuario final.

En términos académicos, el proyecto contribuyó a reforzar conocimientos en **programación orientada a objetos, conexión a bases de datos, patrones de diseño y metodologías ágiles**, lo que representa un aprendizaje integral en el área del desarrollo de software. Finalmente, se concluye que este tipo de aplicaciones son altamente pertinentes en la actualidad, ya que ofrecen una alternativa eficiente y práctica para la gestión de soporte técnico en entornos empresariales y educativos.